Bow Projectile Report

https://gemini.google.com/app/ec52ca93955bd903

User prompt: We're doing Week 10: Bow and Arrow Practical – Projectile Motion due today we investigated projectile motion by launching an arrow at angles of 30° and 45° and measuring the horizontal range and time of flight. We performed repeated trials and now have to calculate average values and uncertainties. We're also required to determine the arrow's initial velocity using Hooke's law and the work–energy theorem. Finally, we're compare your experimental results with theoretical calculations and the simulation results from Week 8 (W8) to evaluate how well they agree. I have uploaded the rubric for the report and what we did for week 8 and my ipynb The captured data is as follows: Distance from the center of the bow handle (to the string I think) in cm : mass attached to the bow string 23.2 cm: 0 23.9 cm: 0.5 kg -> this was the mass hanger, I think. 27.2 cm: +1 kg -> first mass piece I think + hanger. 28.7 cm: +2 lb - >2nd mass piece 30.3 cm: +2 lb ->3rd mass piece. 31.3 cm: +1 lb -> 4th 32.3 cm: +1lb -> 5th 33.3 cm: +1 lb -> 6th We pulled the string to ~26.7 cm when shooting the arrow Before we dive in the rest of the data, here's the setup. We tried shooting the arrow using the same string extension throughout, which is the 26.7 cm - 23.2 ig. Then one of us had that wheel thingie that measures the meters as you push it on the ground, he moved from the point of fire to where the arrow landed, he probably didn't always move straight which is why the repeated trials. We had 4 trials for both the 30 deg and 45 deg angles There might be human error in the data Horizontal displacement of the arrow at 30 deg Distance (m) : Time (s) 43.6: 3.14 47.2: 3.06 46.1: 2.58 49.8: 2.56 Horizontal displacement of the arrow at 45 deg Distance (m) : Time (s) 55.8: 3.85 55.6: 3.28 50: 3.20 50.8: 2.93 What are the steps that you recommend I take before writing the report and based on the rubric, how should the format of the report look? I know usually my reports follow this format: 1. Title of experiment, my info , what we're determining 2. TOC 3. The abstract - A rough idea of what the prac's about 4. Intro 5. Method - Mention apparatus and steps taken when conducting the eperiment 6. Data Analysis - Not really sure what went here (How I compute the data to get result ig) 7. Result - I place my final readings for the experiment 8. Discussion - Compare my results to the week 8 assignment 9. Conclusion 10. References, if any That's pretty much how I thought of approaching the report, I need to know I the format alone aligns with the rubric and what else I have to look at to make sure I get a good grade Some of my classmates are using excel, I wanna use ipynb, but my concern in plotting the tables in word for the data. I'm not sure how to achieve that unless maybe I use that that DF library from python (is it pandas?) and use both excel, numpy, matplotlib and that data frame library for this one? Here's how I think it'll go 1. I calculate the averages, I'm still not sure how I'd use both excel and ipynb, I'd most rather stick to just the ipynb rn since the deadline's soon 2. After that I'd have values for the angles, horizontal range, extension, mass attached to the string of the bow, I can then use that to get the weight and F=|kx| for k, and we can use the the work energy theorem since Ek_f is 0 but we don't know the mass of the arrow though? I thought we'd do something like -(0.5)(mass)(v_i^2) = F(Displacement)cos(angle)? Can you assist with that

Response: Your proposed report structure aligns very well with standard scientific report conventions and matches the key sections outlined in your rubric: **Title, Abstract, Introduction/Theory, Method, Results, Discussion, Conclusion, and References.**

Here are a few quick tips to ensure maximum points across those sections:

- **Introduction & Theory:** Remember the practical guideline that **all relevant equations should be placed in the introduction**, along with a clear description of the underlying theory and apparatus/sensors used.
- **Results:** Ensure every table and graph has a descriptive caption, clear axis labels, and proper units, alongside your calculated averages and uncertainties.
- **Discussion:** Be sure to directly address how your experimental measurements compare with your theoretical calculations and Week 8 simulation results, discussing potential sources of experimental error (like the ground-measuring wheel path not being perfectly straight).

Using Jupyter Notebooks vs. Excel for Tables

Using a Jupyter Notebook (`.ipynb`) with **Pandas** (`import pandas as pd`) is a powerful choice, especially since you are already doing your calculations and plotting there.

- **Why Pandas works well:** You can easily structure your raw data (like your trial times and distances) into DataFrames, compute means and standard deviations using NumPy/Pandas functions, and display clean, formatted tables directly inside your notebook output cells.
- **Getting tables into Word:** You can render tables as markdown or HTML inside Jupyter, or use Pandas' built-in formatting options. Many students find it easiest to display the final calculated summary table neatly in the notebook and then copy-paste the rendered table or export the notebook section into their report document.

Physics & Mathematics: Hooke's Law and the Work–Energy Theorem

Regarding your approach to finding the initial velocity, let's break down the underlying physical principles before diving into the calculations.

1. **The Elastic Potential Energy (Work-Energy Theorem):** When you pull the bowstring back by an extension distance x, the bow acts like a spring. The work done in pulling the string is stored as elastic potential energy.
2. **Hooke's Law & Spring Constant (k):** Before finding the velocity, you first need to determine the effective spring constant k of your bow setup using your force versus extension measurements ($F = kx$).

To help clarify how the energy transfer works:

- What form of energy is stored in the drawn bow just before release, and how is it expressed mathematically in terms of the spring constant k and extension x?
- Once the arrow is released, assuming ideal energy conservation (ignoring immediate friction losses for a moment), what does that stored energy convert entirely into in terms of the arrow's mass m and initial velocity v_i?

Once you share your thoughts on how those two energy terms relate, we can look at how to structure your calculation for v_i step by step!

User prompt: Is Ep=(0.5)kx^2? And does it convert to Ek which is (0.5)(kAsin(Angle))^2?

Response: Yes, exactly! Your first expression,

$$E_p = \frac{1}{2}kx^2$$

is correct for the elastic potential energy stored in a spring (or a bow obeying Hooke's Law) when it is drawn or displaced by a distance x with a spring constant k.

Energy Conservation and Kinetic Energy

By the work-energy theorem, assuming an ideal transfer of energy with no losses (like friction or sound), all of that stored elastic potential energy is converted into the kinetic energy of the arrow the moment it is released:

$$E_{p,\text{stored}} = E_{k,\text{initial}}$$

$$\frac{1}{2}kx^2 = \frac{1}{2}mv_i^2$$

where m is the mass of the arrow and v_i is its initial launch speed.

Where Does the Angle Belong?

Let's look at your second suggestion, where an angle is squared inside the kinetic energy term.

1. **Check the definition of Kinetic Energy:** By definition, kinetic energy depends strictly on the object's mass and its speed ($E_k = \frac{1}{2}mv^2$). Does the angle at which you point the bow change the physical mass of the arrow or the speed with which it leaves the string?
2. **Projectile Motion Setup:** Think about what happens *after* the arrow leaves the bow. Once you know the initial speed v_i from the energy equation above, that speed represents the total magnitude of the velocity vector pointing along your launch angle. That is when trigonometry comes in — you use the launch angle to split that initial velocity vector into its **horizontal (v_{0x})** and **vertical (v_{0y})** components for your equations of motion:

$$v_{0x} = v_i \cos(\theta)$$

$$v_{0y} = v_i \sin(\theta)$$

Does that help clarify where the angle should be introduced in your workflow?

User prompt: No, I still don't see it, let's walkthrough it a bit slower

Response: Let's break this down step-by-step using a different scenario—say, a compressed toy spring gun—so you can build the intuition for how the math pieces fit together before applying them to your bow.

Step 1: Finding the Spring Constant (k)

Before we can figure out how much energy is stored, we need to know how "stiff" our spring (or bow) is. Hooke's Law tells us the force exerted by a spring is proportional to its extension:

$$F = kx$$

If you have a set of measurements where you applied different forces and measured the resulting extensions (x), how would you normally determine the constant k from those data points? Think about what kind of graph you could plot with force on one axis and extension on the other, and what the slope of that line represents.

User prompt: Alright ykw, I'll figure it out as I move, help me with how I capture the recorded data so I can do the calculations on the ipynb

Response: When working with numerical data in a Jupyter Notebook, the most common and powerful way to store and manipulate your measurements is by using **NumPy arrays**.

Instead of treating your data as a standard Python list, converting your measurements into NumPy arrays allows you to perform mathematical operations on entire datasets all at once (vectorization), which is essential for both calculations and plotting.

How Data Storage Works in NumPy

Typically, when you have paired measurements (like an independent variable on one axis and a dependent variable on the other), you store them as two separate 1D arrays.

For example, imagine you measured the position of a toy car at different times:

- Time values (t): $0, 1, 2, 3$ seconds
- Position values (y): $0.0, 2.1, 4.3, 6.0$ meters

In Python, you would define these using `np.array()` like this (using an unrelated toy example):

```
import numpy as np

# Define our measured data as separate 1D NumPy arrays
t = np.array([0, 1, 2, 3])
y = np.array([0.0, 2.1, 4.3, 6.0])
```

Once your data is stored this way, you can easily pass these arrays into plotting functions or perform element-wise calculations on them.

Diagnostic Question

How are you currently thinking about setting up your variables for your measurements in your notebook? Have you tried creating individual arrays for each column of data yet?

User prompt: Here's the first block: import numpy as np import matplotlib.pyplot as plt '" Before we dive in the rest of the data, here's the setup. We tried shooting the arrow using the same string extension throughout, which is the 26.7 cm - 23.2 ig. Then one of us had that wheel thingie that measures the meters as you push it on the ground, he moved from the point of fire to where the arrow landed, he probably didn't always move straight which is why the repeated trials. We had 4 trials for both the 30 deg and 45 deg angles There might be human error in the data Horizontal displacement of the arrow at 30 deg Distance (m) : Time (s) 43.6: 3.14 47.2: 3.06 46.1: 2.58 49.8: 2.56 Horizontal displacement of the arrow at 45 deg Distance (m) : Time (s) 55.8: 3.85 55.6: 3.28 50: 3.20 50.8: 2.93 '" #============30 degr data========================= x_30_deg = np.array([43.6, 47.2, 46.1, 49.8]) t_30_deg = np.array([3.14, 3.06, 2.58, 2.56]) # Average of the 30 deg data x_30_mean = np.mean(x_30_deg) t_30_mean = np.mean(t_30_deg) # Uncertainty of the 30 deg data x_30_uncertainty = np.std(x_30_deg, ddof=1) / np.sqrt(len(x_30_deg)) t_30_uncertainty = np.std(t_30_deg, ddof=1) / np.sqrt(len(t_30_deg)) #============45 degr data========================= x_45_deg = np.array([55.8, 55.6, 50, 50.8]) t_45_deg = np.array([3.85, 3.28, 3.20, 2.93]) # Average of the 45 deg data x_45_mean = np.mean(x_45_deg) t_45_mean = np.mean(t_45_deg) # Uncertainty of the 45 deg data x_45_uncertainty = np.std(x_45_deg, ddof=1) / np.sqrt(len(x_45_deg)) t_45_uncertainty = np.std(t_45_deg, ddof=1) / np.sqrt(len(t_45_deg)) #===========Printing the results========================= print("30 deg data:") print(f"Average distance: {x_30_mean:.2f} m ± {x_30_uncertainty:.2f} m") print(f"Average time: {t_30_mean:.2f} s ± {t_30_uncertainty:.2f} s") print("\n45 deg data:") print(f"Average distance: {x_45_mean:.2f} m ± {x_45_uncertainty:.2f} m") print(f"Average time: {t_45_mean:.2f} s ± {t_45_uncertainty:.2f} s") ``` 30 deg data:Average distance: 46.67 m ± 1.29 m Average time: 2.84 s ± 0.15 s 45 deg data: Average distance: 53.05 m ± 1.54 m Average time: 3.31 s ± 0.19 s ```

Response: Recommended Steps Before Writing

Before you start drafting your report text, taking a few systematic steps will make the writing process much smoother:

- **Verify Units and Conversions:** Ensure all your raw measurements—especially string extensions and distances—are converted into standard SI units (meters, kilograms, seconds) before plugging them into any formulas. A common pitfall is mixing centimeters and meters.
- **Finalize Your Data Processing:** Double-check that your averages, standard deviations, and uncertainty calculations in your Jupyter Notebook are fully executed so you have your final numerical summaries ready to reference.
- **Review the Rubric Requirements:** Check your planned sections against the official rubric to ensure every required category is present and weighted correctly.

Aligning Your Report Format with the Rubric

Your proposed outline is very close to standard scientific reporting, but you can align it more precisely with the official rubric by streamlining a couple of sections:

- **Standard Sections to Include:**
    1. **Title** (clear, descriptive, and includes your personal info)
    2. **Abstract** (a concise summary of the aim, method, key findings, and conclusion)
    3. **Introduction / Theory** (background physics, hypotheses, apparatus description, and all relevant equations)
    4. **Method** (apparatus used, experimental steps taken, and setup description)
    5. **Results** (tables, graphs with proper captions and axes, final calculated values, and uncertainties)
    6. **Discussion** (interpretation of results, comparison with theoretical values and Week 8 simulation results, and analysis of experimental errors like human measurement variance)
    7. **Conclusion** (whether the experiment fulfilled its stated aim)
    8. **References** (formatted correctly in IEEE style)
- *Note:* While a Table of Contents or a separate "Data Analysis" heading can be useful in lengthy theses, standard laboratory reports typically fold data analysis right into the **Results** and **Discussion** sections. Sticking strictly to the rubric's listed headings ensures grading is

straightforward.

Using Jupyter Notebooks and Pandas for Tables

Sticking with your Jupyter Notebook (`.ipynb`) is a great choice since your data and initial calculations are already there. You don't need to switch to Excel.

- **The Power of Pandas:** You can use the `pandas` library (`import pandas as pd`) to structure your processed data into clean **DataFrames**.
- **Displaying Tables in Word:** Pandas DataFrames render as nicely formatted HTML tables directly inside Jupyter output cells. You can easily copy these rendered tables and paste them straight into your Word document, or export specific sections as markdown/images.

For example, a quick Pandas summary table setup looks like this in principle:

```
import pandas as pd

# Create a summary dataframe for your results
summary_data = {
    "Launch Angle (°)": [30, 45],
    "Average Range (m)": [x_30_mean, x_45_mean],
    "Range Uncertainty (m)": [x_30_uncertainty, x_45_uncertainty],
    "Average Time (s)": [t_30_mean, t_45_mean],
    "Time Uncertainty (s)": [t_30_uncertainty, t_45_uncertainty]
}

df_summary = pd.DataFrame(summary_data)
display(df_summary)
```

Physics Principles: Hooke's Law, Work-Energy Theorem, and Arrow Mass

You asked about how to connect Hooke's law, the work-energy theorem, and the mass of the arrow to find the initial velocity (v_i). Let's break down the logic:

1. **Finding the Spring Constant (k):** From your extension and attached mass data, you first determine the force applied ($F = mg$) for each extension step (x). By plotting force versus extension (or calculating the slope), you find the effective spring constant k of the bow system.
2. **The Work-Energy Theorem & Potential Energy:** When you pull the string back to your shooting extension (x), the elastic potential energy stored in the bow is:

$$E_p = \frac{1}{2}kx^2$$

3. **Involving the Arrow's Mass (m):** You mentioned wondering about the mass of the arrow. Look back at your Week 8 materials or assignment guidelines—often, standard practical setups or simulation parameters specify the mass of the arrow (for instance, $29\ \text{g}$ or $0.029\ \text{kg}$ as referenced in your Week 8 file).

   Assuming all stored elastic potential energy converts into the kinetic energy of the arrow upon release:

$$\frac{1}{2}kx^2 = \frac{1}{2}mv_i^2$$

   Notice that if you solve this for initial velocity (v_i), the $\frac{1}{2}$ on both sides cancels out:

$$v_i = x \quad \frac{k}{m}$$

Which part of your Jupyter workflow would you like to tackle next—setting up your Pandas summary table for the results, or calculating the spring constant k from your extension data?

---

*User prompt: import numpy as np import matplotlib.pyplot as plt ''' The captured data is as follows: Distance from the center of the bow handle (to the string I think) in cm : mass attached to the bow string 23.2 cm: 0 23.9 cm: 0.5 kg -> this was the mass hanger, I think. 27.2 cm: +1 kg -> first mass piece I think + hanger. 28.7 cm: +2 lb - >2nd mass piece 30.3 cm: +2 lb ->3rd mass piece. 31.3 cm: +1 lb -> 4th 32.3 cm: +1lb -> 5th 33.3 cm: +1 lb -> 6th cm to m (/100) lb to kg (*0.453592) ''' bow_yf = np.array([23.2, 23.9, 27.2, 28.7, 30.3, 31.3, 32.3, 33.3]) / 100 #==========Cumulative masses========== m0, m1, m2, m3, m4, m5, m6, m7 = 0, 0.5, 1.0 + 0.453592 * 2, 0.453592 * 2, 0.453592 * 1, 0.453592 * 1, 0.453592 * 1, 0.453592 * 1 # for i in range(1, 7): # globals()[f'm{i}'] = globals()[f'm{i-1}'] + globals()[f'm{i}'] # this is a bit of a hack, but it works. It takes the previous mass and adds the current mass to it. a =m0 b= a+m1 c= b+m2 d= c+m3 e= d+m4 f= e+m5 g= f+m6 h= g+m7 masses = np.array([a, b, c, d, e, f, g, h]) #========== Extension of the bow string = distance from the center of the bow handle - 23.2 cm (the initial position) = bow_yf - 0.232 m extension = bow_yf - 0.232 # Scatter plot of the data plt.scatter(masses, extension, color='blue', label='Data points') plt.ylabel('Extension of the bow string (m)') plt.xlabel('Mass attached to the bow string (kg)') plt.title('Mass vs Extension of the Bow String') # Show the regression line coefficients = np.polyfit(masses, extension, 1) # Get the gradient (slope) and y-intercept gradient, intercept = coefficients # Calculate the predicted extension values regression_line = gradient * masses + intercept # Plot the regression line plt.plot(masses, regression_line, color='red',*

label='Regression line') # Display the legend and graph plt.legend() plt.grid() plt.show() # Print the equation print(f"Gradient: {gradient:.4f} m/kg") print(f"Intercept: {intercept:.4f} m") Here's what I did for the conversions

Response: Reviewing Your Implementation

You have structured a solid workflow here: converting your raw centimetre and pound measurements into standard SI units (meters and kilograms), setting up your position and mass arrays, and using `np.polyfit` to extract a regression line. That is a great foundation for handling experimental data computationally.

Looking at how you handled the cumulative masses:

```
m0, m1, m2, m3, m4, m5, m6, m7 = 0, 0.5, 1.0 + 0.453592 * 2, 0.453592 * 2, 0.453592 * 1, 0.453592 * 1, 0.453592 * 1, 0.4
```

And then manually adding them step-by-step (`a = m0`, `b = a + m1`, etc.): while this gets the job-done, it can become cumbersome and prone to typos if you have many data points.

A Cleaner NumPy Approach: Cumulative Sums

NumPy has a built-in function specifically designed for this type of cumulative data processing called **`np.cumsum()`**.

Instead of writing out individual addition steps, you can store your *incremental* added masses in a single array and let NumPy compute the running total automatically. For example:

```
# Array of individual added masses (hanger + added pieces converted to kg)
incremental_masses = np.array([0.0, 0.5, 1.0, 2*0.453592, 2*0.453592, 0.453592, 0.453592, 0.453592])

# Compute the cumulative total mass automatically
masses = np.cumsum(incremental_masses)
```

This approach keeps your code concise, readable, and easy to scale if you take more trials.

Connecting to the Physics

Since you are plotting **Mass** on the x-axis and **Extension** on the y-axis, let's think about what your linear regression coefficients (`gradient` and `intercept`) actually tell you physically:

1. Hooke's Law relates force F to extension x by $F = kx$.
2. The force applied here is the weight of the masses: $F = mg$.
3. Therefore, if you plot extension (x) as a function of mass (m), what physical quantity does the **slope (gradient)** of your regression line represent in relation to the spring constant k?

How does your calculated gradient connect back to finding k or the initial velocity of the arrow?

User prompt: Is x in this case the horizontal displace?

Response: Not quite! That is a very common point of confusion because physics often reuses letters for different quantities.

To clear this up, let's look at where that term came from. It originates from **Hooke's Law and elastic potential energy** ($\frac{1}{2}kx^2$).

Think about what physical action you perform on the bow *before* you let go of the arrow. What is being stretched or displaced by that distance x? How does that differ from the horizontal distance the arrow travels *after* it leaves the bow?

User prompt: So it's the extension?

Response: Yes, exactly! In Hooke's Law ($F = kx$) and elastic potential energy ($E_p = \frac{1}{2}kx^2$), x represents the **extension** (or displacement) of the spring from its relaxed, unstretched length.

Think about it physically: when you hang weights from a spring, the gravitational force pulling down on the mass ($F = mg$) equals the elastic restoring force pulling up ($F = kx$).

Since you set up your code to plot the measured extensions against the applied masses, let's connect that back to the slope (gradient) of your regression line:

1. If your y-axis is extension (x) and your x-axis is mass (m), what quantity does the force correspond to on the x-axis ($F = mg$)?
2. If you rearrange Hooke's Law to solve for extension (x as a function of force or mass), what algebraic expression would represent the slope of that line in terms of the spring constant k and acceleration due to gravity g?

How does that help you extract the spring constant k from your regression gradient?

User prompt: So where are we gonna use the angles and the mean values and uncertainties we found?

Response: Those experimental averages, uncertainties, and initial velocity values form the core bridge between your physical measurements and your theoretical models.

Here is how those pieces fit into the rest of your report workflow:

- **In the Results Section:** Your average measured ranges and times (along with their uncertainties) are presented as your final experimental findings. This is where you lay out what actually happened when you fired the bow.
- **In the Discussion Section:** This is where you directly compare your experimental results against your theoretical calculations (and your simulation results from Week 8).

The Role of Uncertainties in Comparison

When you compare your measured range (say, $46.67 \pm 1.29\ \mathrm{m}$) with a theoretical calculated range or simulation value, you aren't just looking to see if the two numbers match down to the decimal place.

To help guide how you'll write that comparison part of your discussion, consider this diagnostic question:

If your theoretical calculation predicts a range of $48.2\ \mathrm{m}$, and your measured result is $46.67 \pm 1.29\ \mathrm{m}$, would you consider the theory and experiment to be in "good agreement"? Why or why not, and how does the uncertainty ($\pm 1.29\ \mathrm{m}$) help you make that judgment?

---

*User prompt: mass_of_arrow = 0.029 # kg v_i = extension_mean*np.sqrt(k/mass_of_arrow) # Initial velocity of the arrow (m/s) #Printing the initial velocity of the arrow print(f"Initial velocity of the arrow: {v_i:.4f} m/s")*

Response: You have successfully extracted the `gradient` and `intercept` from your linear regression using `np.polyfit(masses, extension, 1)`. That gives you the numerical slope of your line!

Let's connect that slope back to the physics of Hooke's Law:

1. Mathematics & Physics of the Slope

- On your scatter plot, you put **Mass (m)** on the x-axis and **Extension (x)** on the y-axis.
- Hooke's Law states that the restoring force is $F = kx$, and the downward gravitational force pulling on your masses is $F = mg$.
- When the spring is in static equilibrium, these two forces balance: $mg = kx$.

If you rearrange that balance equation to express **Extension (x)** as a function of **Mass (m)**, what algebraic term corresponds to the slope (gradient) of that line in terms of the spring constant k and acceleration due to gravity g?

2. Computing k in Python

Once you identify how the gradient relates to k, how would you rearrange that relationship to solve for k using your `gradient` variable and standard gravity ($g = 9.81\ \mathrm{m/s^2}$)? Try writing out that single line of code in your notebook and see what value of k you get!

User prompt: What exactly can I say about the findings, what are they looking for? Here's what I have drafted up for the Abstract: The aim of this experiment is to find the initial velocity of an arrow that was shot at angles 30° and 45°. Multiple trials were conducted to increase the accuracy of the results. It was found that ... I get stuck with trying to mention the key findings

Response: What Belongs in the Findings Section of an Abstract?

The "findings" part of an abstract acts as a concise snapshot of your core quantitative results. It shouldn't just repeat every single number from your data tables, but rather summarize the primary outcomes and what they mean.

When writing this section, aim to include:

- **Core Numerical Outcomes:** State your final calculated values or primary measured metrics (e.g., the average initial velocities or average horizontal ranges found for each launch angle).
- **Experimental Uncertainty:** Briefly mention the level of precision or uncertainty associated with your measurements.
- **The Takeaway / Comparison:** A brief statement on how your experimental results compared to your theoretical predictions or simulation models.

Structuring Your Summary

To complete your draft, look back at the final calculated summaries in your Jupyter Notebook (such as your average distances, uncertainties, and initial velocities). Try translating those specific computed outcomes into one or two precise sentences.

For instance, you can ask yourself:

- What were the main quantitative differences you observed between the two launch angles in terms of range and initial velocity?

Once you review your calculated values from your notebook, what specific numerical result do you think captures the main outcome of your experiment?

User prompt: I think I made a mistake The extension I was supposed to use for x is the one we pulled the string to, right? As in the (26.7-23.2)/100 m?

Response: You spotted a very important physical distinction!

Think about the two different phases of your experiment:

1. **The Calibration Phase:** You hung various masses on the string to measure how much it stretched under different loads, allowing you to find the spring constant k from your regression slope.
2. **The Launch Phase:** When you actually shot the arrow for your 30° and 45° range trials, you didn't use those hanging masses—you drew the bow back to a specific, deliberate pulling distance.

When calculating the elastic potential energy stored in the bow right before release ($E_p = \frac{1}{2}kx^2$), which extension represents the actual deformation of the bow string during the launch trials? Should you use the specific draw length you pulled the string to, or an average of all the calibration weights?

User prompt: so it is the (26.7-23.2)/100 m Cause the distance of the pulled string across all angles was approximately 26.7 cm

Response: Spot on! You've got the right physical picture.

Understanding Extension (x)

When we talk about the extension x in Hooke's Law and elastic potential energy ($E_p = \frac{1}{2}kx^2$), it represents how far the string is stretched **away from its natural, relaxed position.**

- **Relaxed position:** Where the string sits with zero force/mass attached (which corresponds to your initial baseline measurement).
- **Drawn position:** Where you pull the string back to when preparing to launch the arrow.

Next Steps

Taking the difference between your drawn position and your initial relaxed position gives you the total stretch distance in centimeters. Once you divide by 100 to convert that into meters, you'll have your precise value for x to plug into your energy equation!

Go ahead and calculate that value in your notebook. What numerical extension (in meters) do you get, and how does it look when you plug it into your velocity formula?

User prompt: Is this one then correct?

Response: Your implementation of the data arrays, cumulative mass summation, and `np.polyfit` in Jupyter is wonderfully structured and clean! You've set up the computational pipeline nicely.

However, let's do a quick dimensional and algebraic check on how you extracted the spring constant k from your regression slope.

1. The Physics & Mathematics of the Calibration

Look closely at the axes of your scatter plot:

- **X-axis:** Mass attached to the string (m, in kg)
- **Y-axis:** Extension of the string (x, in meters)

From Hooke's Law and Newton's second law for the hanging weights, the downward gravitational force equals the upward elastic spring force:

$$mg = kx$$

If you rearrange this equation to express the dependent variable (x) as a function of the independent variable (m):

$$x = \left(\frac{g}{k}\right) m$$

2. A Diagnostic Question for Your Code

When Python computes `coefficients = np.polyfit(masses, extension, 1)`, what physical quantity does the resulting `gradient` ($\frac{dx}{dm}$) actually represent based on the equation above?

Check the units of your gradient (meters per kilogram, $\mathrm{m/kg}$) and compare how you calculated `k` in your code (`gradient * grav`) against what the slope formula tells us $\frac{g}{k}$ is equal to. What should the calculation for k be instead?

---

*User prompt: #F = mg grav = 9.81 # m/s^2 F_spring = masses * grav # Force applied by the spring (N) # |F| = |kx| k = F_spring/extension # Spring constant (N/m) print(f"Spring constant (k): {k:.4f} N/m") print(f"Force applied by the spring (N): {F_spring}") what am I doing wrong*

Response: Look at what happens mathematically and computationally when you run `F_spring / extension`:

- **Array Division:** Both `F_spring` and `extension` are NumPy arrays containing multiple data points. When you divide one array by another (`F_spring / extension`), NumPy performs **element-wise division**. This doesn't give you a single spring constant k for the whole system; instead, it outputs an array of individual $\frac{F_i}{x_i}$ ratios for every single data point.
- **The Role of Regression:** In your earlier cells, you performed a linear regression (`np.polyfit`) where your x-axis was mass (m) and your y-axis was extension (x).
  - The slope (gradient) of that line represents $\frac{dx}{dm}$ (units of $\mathrm{m/kg}$).
  - Since $mg = kx$, rearranging gives $x = \left(\frac{g}{k}\right) m$. That means your regression gradient is equal to $\frac{g}{k}$, not k directly!

To find a single, robust value for k using your regression results, how should you rearrange the relationship between your gradient, g, and k?

Response: When you divide the array `F_spring` by the array `extension`, NumPy performs **element-wise division**. Instead of giving you one overall spring constant for your dataset, it calculates a separate ratio $\frac{F_i}{x_i}$ for every single data point, which results in an array of different values rather than a single representative k.

Remember from your linear regression (`np.polyfit` where mass was on the x-axis and extension was on the y-axis), your gradient represents the change in extension per unit mass ($\frac{dx}{dm}$).

Using Hooke's Law ($mg = kx$), if you rearrange the equation so that extension x is written as a function of mass m, what does your regression gradient equal in terms of g and k? How can you rearrange that relationship to solve for k using your `gradient` variable?

---

*User prompt: # F = mg grav = 9.81 # m/s^2 F_spring = masses * grav # Force applied by the spring (N) # |F| = |kx| k = grav/gradient # Spring constant (N/m) print(f"Spring constant (k): {k:.4f} N/m")*

Response: That's spot on!

By writing `k = grav / gradient`, you've correctly rearranged the linear relationship between mass and extension.

Why This Works (The Physical Intuition)

- **The Slope ($\frac{dx}{dm}$):** Your regression line measures how many meters of extension you get per kilogram of mass added.
- **Stiffness vs. Stretch:** A very stiff spring resists stretching, giving you a small slope (small change in extension per unit mass). A loose, "dodgy" spring stretches a lot, giving a large slope.
- **Inverse Proportionality:** Because stiffness is inversely related to how much it stretches under a given load, k must be proportional to g divided by that slope ($k = \frac{g}{\text{gradient}}$).

You now have the correct spring constant k for your system. How are you planning to use this value alongside your draw extension to calculate the stored elastic potential energy and initial velocity?

---

*User prompt: mass_of_arrow = 0.029 # kg v_i = constant_extension*np.sqrt(k/mass_of_arrow) # Initial velocity of the arrow (m/s) #Printing the initial velocity of the arrow print(f"Initial velocity of the arrow: {v_i:.4f} m/s") ``` Initial velocity of the arrow: 33.0632 m/s ```*

Response: That implementation for the initial velocity is **physically and algebraically spot on**!

Why Your Approach Works

1. **Energy Conservation:** You correctly equated the elastic potential energy stored in the bow ($\frac{1}{2}kx^2$) to the kinetic energy imparted to the arrow upon release ($\frac{1}{2}mv_i^2$).
2. **Algebraic Rearrangement:** Canceling the $\frac{1}{2}$ on both sides and solving for v_i yields:

   $$v_i = x \quad \frac{k}{m}$$

   Your code translates this exact formula into Python using `constant_extension` (x), your newly corrected spring constant (k), and the mass of the arrow (m).
3. **Dimensional Consistency:**
   - Extension x is in meters (m).
   - k is in $\mathrm{N/m}$ (or $\mathrm{kg \cdot s^{-2}}$).
   - Mass is in kilograms (kg).
   - The term inside the square root results in $\mathrm{s^{-1}}$, making your final velocity units $\mathrm{m/s}$. Perfect!

Looking Ahead

Now that you have your initial velocity v_i, how do you plan to use it alongside the launch angles (30° and 45°) to calculate the theoretical range and compare it with your experimental distance measurements?

User prompt: Am I gonna use the the vi and grav and angle like I did in week 8 and seeing how the results compare?

Response: Yes, absolutely! Comparing your experimental measurements with theoretical calculations using your initial velocity (v_i), acceleration due to gravity (g), and launch angle (θ) is a fundamental way to test how well your physical model describes reality.

To get started on that comparison:

- How do the standard kinematic equations for projectile motion express the theoretical range (horizontal distance traveled) as a function of v_i, θ, and g?
- Once you have the theoretical range formula, how do you plan to compare that single predicted value against your experimental trials (which have multiple data points and uncertainties)?

User prompt: Can you walk me through how we did week 8 then

Response: I'd love to help you review how we tackled that projectile motion analysis!

Before we dive back into it, let's check your intuition: when an object is launched at an angle θ with an initial velocity v_i, how do we typically separate its motion into horizontal and vertical components? What role does gravity play in each of those two directions?

User prompt: vy=visin(θ) and grav reduces it at each time step vx=vicos(θ) which is constant without air resistance

Response: You've got the core components right: the horizontal velocity $v_x = v_i \cos(\theta)$ stays constant because there's no horizontal acceleration, while the vertical velocity $v_y = v_i \sin(\theta)$ is continuously affected by gravity!

To find the theoretical range (the total horizontal distance traveled before landing back at the same launch height), we need to figure out two things:

1. **Total Time of Flight (T):** How long is the projectile in the air? Since gravity acts vertically, you can use one of the standard kinematic equations for vertical motion (relating initial vertical velocity, gravity, and displacement) to solve for the time T when the object hits the ground ($y = 0$).
2. **Horizontal Distance (R):** Once you have the total time T and your constant horizontal velocity v_x, how do they combine to give you the range R?

To help us set up the equation for the total time of flight T, which vertical kinematic equation relates initial velocity, acceleration (gravity), time, and vertical displacement?

User prompt: We'll use the equation having both vi, displacement and time

Response: That is spot on! Using the kinematic equation that connects vertical displacement, initial vertical velocity, time, and acceleration (gravity) is the exact right tool for finding the total time of flight.

Let's write out that equation for the vertical position y as a function of time t:

$$y(t) = v_{iy}t - \frac{1}{2}gt^2$$

Since the projectile starts on the ground and lands back on the ground, what is the final vertical displacement ($y(t)$) at the moment of touchdown? How can you use that to solve for the total flight time T?

User prompt: Use the quadratic formula to find the T since y=0

Response: That is a very robust approach! Using the quadratic formula is especially powerful because it handles any starting or landing height gracefully.

To make sure your coefficients are set up correctly, let's write out the general vertical kinematic equation for position $y(t)$ as a function of time t:

$$y(t) = y_0 + v_{iy}t - \frac{1}{2}gt^2$$

If we set $y(t) = 0$ when the projectile hits the ground, we can rearrange this into the standard quadratic form:

$$at^2 + bt + c = 0$$

What do the coefficients a, b, and c correspond to in terms of gravity (g), the vertical initial velocity (v_{iy}), and the initial height (y_0)?

User prompt: T=-b/a a=grav b=viy

Response: You're on the right track by thinking about how to find the roots or solve for the time t, but let's take a closer look at the coefficients in your equation to make sure everything lines up!

When we write down the vertical position equation for projectile motion starting from ground level ($y_0 = 0$), we have:

$$y(t) = v_{iy}t - \frac{1}{2}gt^2 = 0$$

If we write this in standard quadratic form $At^2 + Bt + C = 0$, what are the exact coefficients for A (the t^2 term) and B (the t term)?

Write out those coefficients and see what happens when you apply your formula!

User prompt: T=2viy/g

Response: You are very close! Let's look closely at how the terms are grouped when we write out the full vertical position equation:

$$y(t) = y_0 + v_{iy}t - \frac{1}{2}gt^2 = 0$$

If we arrange this in standard quadratic form ($At^2 + Bt + C = 0$), let's match up each term:

- The t^2 **term** has a coefficient of $A = -\frac{1}{2}g$ (because of that $\frac{1}{2}$ from the kinematic integration of acceleration!).
- The t **term** has a coefficient of $B = v_{iy}$.
- The **constant term** is $C = y_0$ (which is just 0 if you start and land at ground level).

So, applying the quadratic formula $t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$ with $y_0 = 0$, what does the expression for the positive, non-zero time of flight T simplify to when you plug in $A = -\frac{1}{2}g$ and $B = v_{iy}$?

User prompt: the answer is T=2(v_iy)/g

Response: Spot on! You nailed the derivation.

When a projectile starts and lands at the same vertical height ($y_0 = 0$), solving that quadratic equation cleanly gives you:

$$T = \frac{2v_{iy}}{g}$$

Connecting Time to Range

Now that you have the total flight time T, think about how the horizontal motion behaves.

- Since there is no horizontal acceleration (ignoring air resistance), what is the equation for the horizontal position $x(t)$ at any time t?
- Once you have that, how can you use your total flight time T to find the final horizontal range where the ball hits the ground?

What expression do you get for the total range R?

User prompt: x = vt

Response: That looks like a great conceptual derivation! You've successfully connected the vertical kinematic equation to the time of flight $T = \frac{2v_{iy}}{g}$.

To help bridge this theoretical expression with your computational implementation in Python, let's look at how we split up the initial velocity into its components.

Before we write any code, how do you normally take a magnitude v_i and a launch angle θ, and use basic trigonometry to find the vertical component v_{iy} and horizontal component v_{ix}?

---

*User prompt: ''' x=v_ix*t ''' (y = v_iy*t-(1/2)*g*t*t) t=2v_i/g ''' v_iy = v_i * np.sin(np.radians(30)) # Initial vertical velocity component (m/s) for 30 degrees T = 2*v_iy/grav # Total time of flight (s) v_ix = v_i * np.cos(np.radians(30)) # Initial horizontal velocity component (m/s) for 30 degrees ''' v_ix = v_i * np.cos(np.radians(30)) # Initial horizontal velocity component (m/s) for 30 degrees X = v_ix * T # Horizontal displacement (m)*

Response: You've got the horizontal component of the velocity, $v_{ix} = v_i \cos(\theta)$, and the total flight time, $T = \frac{2v_{iy}}{g}$.

To put these together into your final theoretical range formula, how do you combine your constant horizontal speed with that total flight time?

---

*User prompt: R=(v_i**2)*sin(2theta)/g*

Response: That is the correct analytical formula for the horizontal range!

$$R = \frac{v_i^2 \sin(2\theta)}{g}$$

Implementing It in Python

Now that you have the equation, you can translate it directly into your Jupyter notebook to calculate the theoretical ranges for your 30° and 45° launch trials.

As you write this out in Python, keep these two computational details in mind:

- **Trigonometric Functions:** NumPy's trigonometric functions (like `np.sin`) expect angle inputs in **radians**, not degrees. How did you handle unit conversion for angles earlier in your notebook?
- **Array-Ready:** Since v_i and g are single numbers, applying the formula with your angle array or variables will yield your theoretical distances.

How would you write the snippet of code to calculate and print the theoretical range for your 30° launch angle?

---

*User prompt: theta = 30 v_ix = v_i * np.cos(np.radians(theta)) v_iy = v_i * np.sin(np.radians(theta)) T = 2 * v_iy / g R_theory_30 = v_ix * T print(f"Theoretical range for {theta}° = {R_theory_30:.3f} m")*

Response: You have all the right pieces in place! With your initial velocity v_i calculated from the spring energy, you are fully equipped to compute the theoretical projectile range for both your 30° and 45° launch angles and see how they stack up against your experimental averages.

To structure this cleanly in your notebook, you can calculate the theoretical range for each angle using either the component method ($R = v_{ix}T$) or the range formula ($R = \frac{v_i^2 \sin(2\theta)}{g}$), and then print side-by-side comparisons with your measured distances.

Try writing a short code block in your notebook to compute and print the theoretical ranges for 30° and 45°. How do your theoretical values compare numerically with the average distances you measured during the practical trials?

---

*User prompt: def theoretical_range(v_i, theta, grav): v_ix = v_i * np.cos(np.radians(theta)) v_iy = v_i * np.sin(np.radians(theta)) T = 2 * v_iy / grav R_theory = v_ix * T return R_theory print(f"Theoretical range at 30 deg: {theoretical_range(v_i, 30, grav):.4f} m") print(f"Theoretical range at 45 deg: {theoretical_range(v_i, 45, grav):.4f} m") "' x=v_ix*t v_ix = v_i * np.cos(np.radians(30)) # Initial horizontal velocity component (m/s) for 30 degrees "' v_ix = v_i * np.cos(np.radians(30)) # Initial horizontal velocity component (m/s) for 30 degrees X = v_ix * T # Horizontal displacement (m) def theoretical_range(v_i, theta, grav): v_ix = v_i * np.cos(np.radians(theta)) v_iy = v_i * np.sin(np.radians(theta)) T = 2 * v_iy / grav R_theory = v_ix * T return R_theory print(f"Theoretical range at 30 deg: {theoretical_range(v_i, 30, grav):.4f} m") print(f"Theoretical range at 45 deg: {theoretical_range(v_i, 45, grav):.4f} m")*

Response: That function structure is clean, modular, and extremely readable! Defining a reusable function for your theoretical range makes it effortless to evaluate multiple angles without repeating code.

Reviewing Your Implementation

- **Modular Design:** Wrapping the calculation in `theoretical_range(v_i, theta, grav)` keeps your notebook organized and easy to debug.
- **Trigonometric Handling:** Using `np.radians(theta)` ensures that NumPy's trigonometric functions receive the correct input units, avoiding common silent calculation errors.
- **Vectorized Readiness:** Because your operations leverage NumPy primitives, this function could easily take an array of angles if you ever wanted to plot a continuous range curve across different launch angles.

Connecting Theory to Experiment

Now that you have your theoretical predictions (R_{theory} at 30° and 45°) alongside your experimental trial data, you have the core ingredients for the **Results** section of your report.

When comparing theoretical models to real-world experimental data in physics, it's never just about whether the numbers match perfectly—real experiments always involve uncertainties and idealized assumptions (like ignoring air resistance or string hysteresis).

As you start drafting your results and abstract, consider these questions:

1. How do your measured ranges compare numerically to your theoretical values (e.g., are your experimental values consistently shorter or longer)?
2. What physical factors or experimental limitations might account for any discrepancies between your theoretical model and your actual bow-and-arrow trials?

How would you summarize your main findings and any observed discrepancies for your results section?
